COS20028 – BIG DATA ARCHITECTURE AND APPLICATION

Dr Pei-Wei Tsai (Lecturer, Tutor, and Unit Convenor)
ptsai@swin.edu.au, EN508d

SWINBURNE UNIVERSITY OF TECHNOLOGY

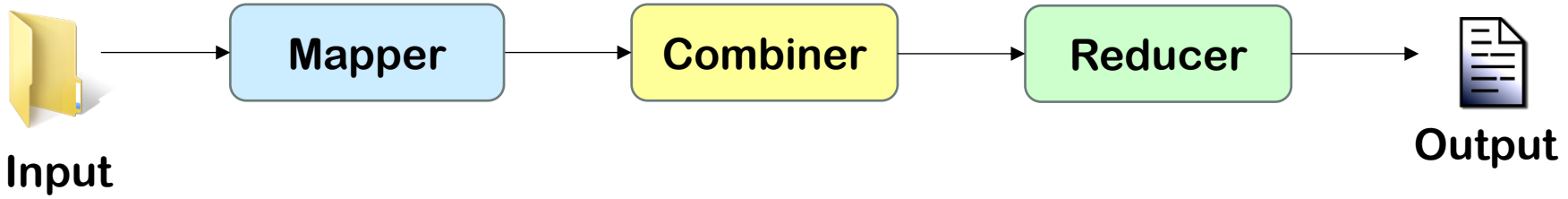
Week 6

Partitioners and Data Format

PARTITIONER

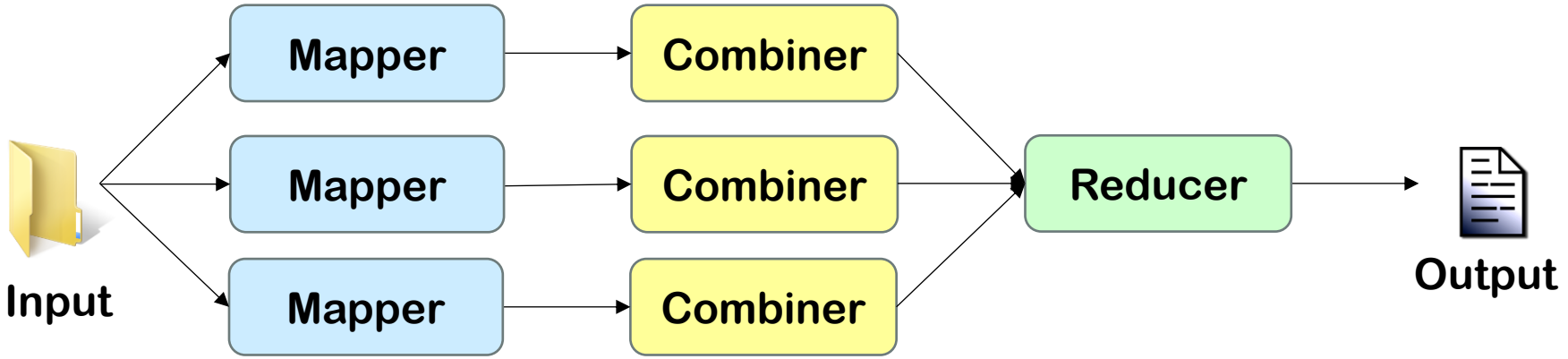
MapReduce Pipeline

No. of Mapper: 1
No. of Combiner: 1
No. of Reducer: 1



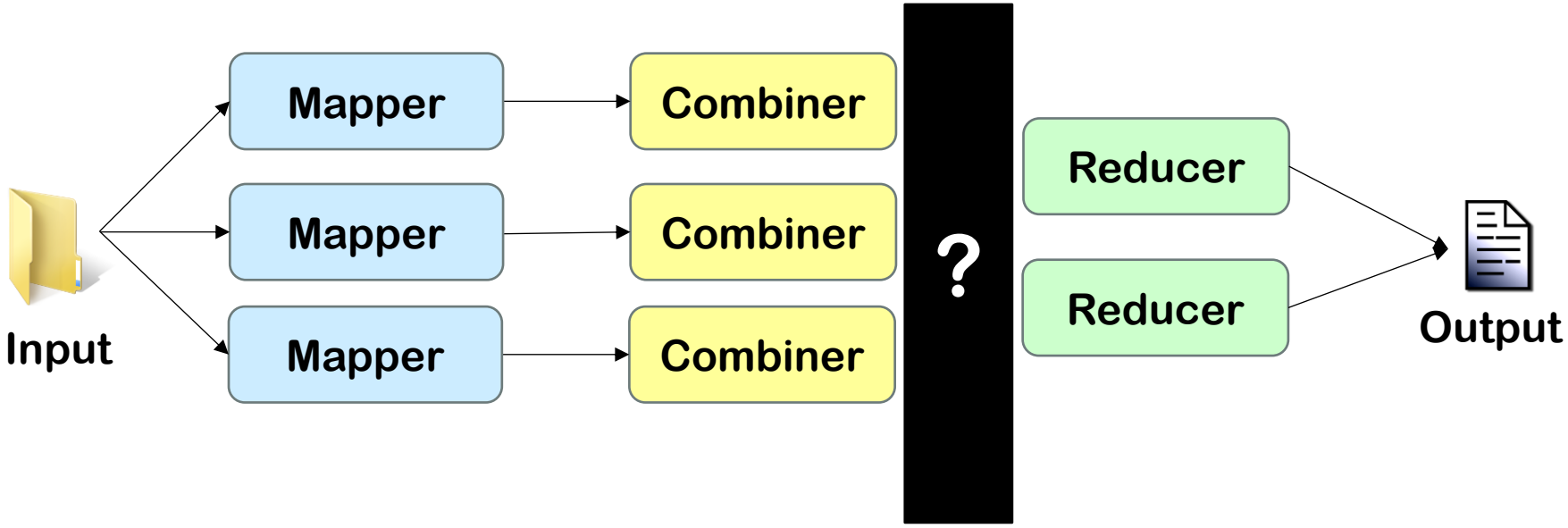
MapReduce Pipeline

No. of Mapper: 3
No. of Combiner: 3
No. of Reducer: 1



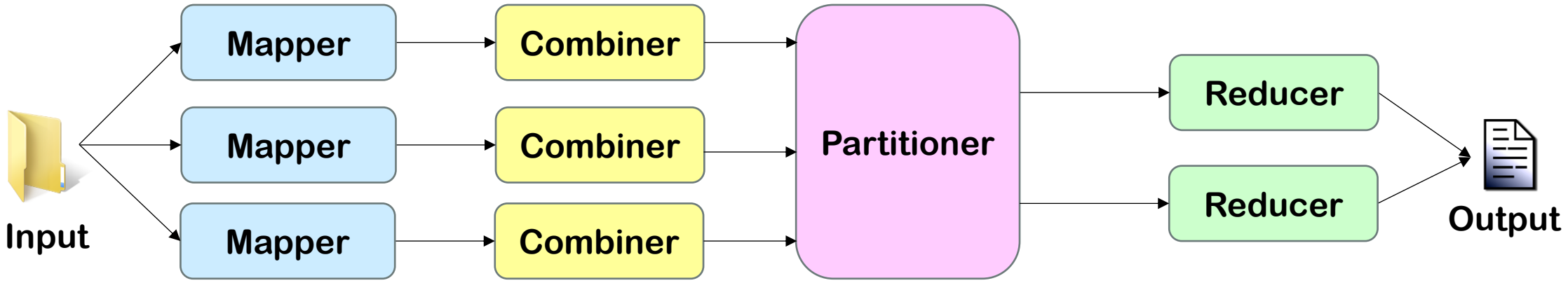
MapReduce Pipeline

No. of Mapper: 3
No. of Combiner: 3
No. of Reducer: 2



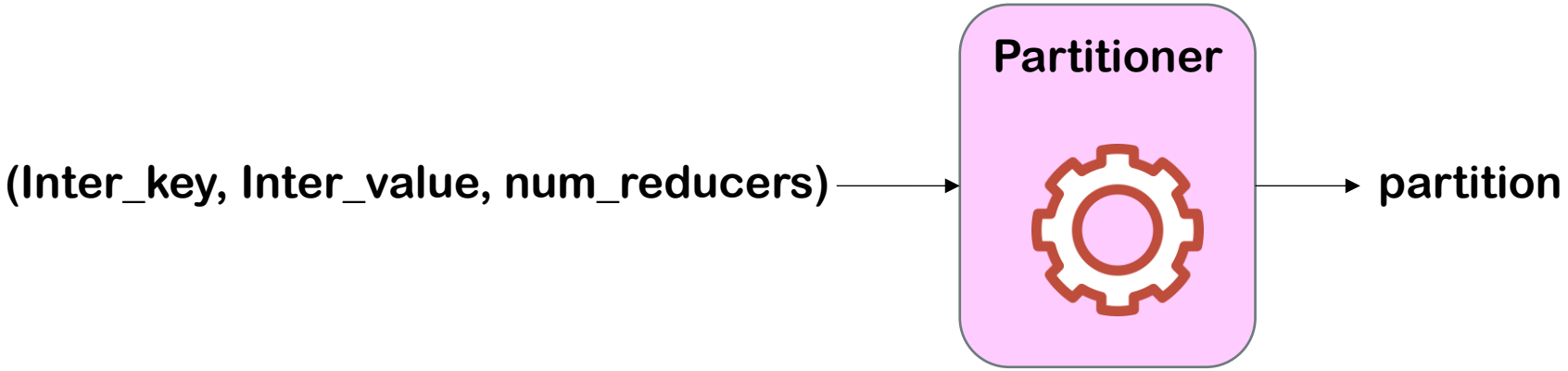
MapReduce Pipeline

No. of Mapper: 3
No. of Combiner: 3
No. of Reducer: 2



- The Partitioner determines which Reducer each intermediate key-value pair goes to.

The Typical Outlook of the Partitioner



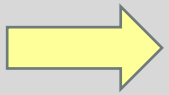
The Default Partitioner

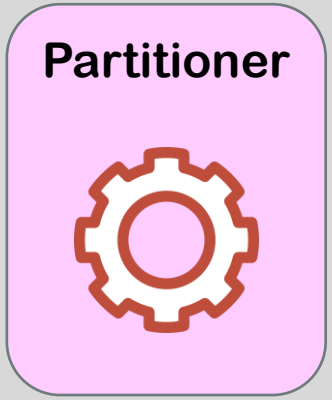
- The default Partitioner is the *HashPartitioner*
  - It uses the java *hashCode* method
  - Guarantees all pairs with the same key go to the same Reducer.
- Hash
  - The output of the hash function from the same input key is always the same.

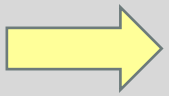
Example of a Hash Partitioner

Output from Mapper

Key	Value
Cat	1
Dog	1
Cat	1
Pig	1
Bird	1







Key	Value
Bird	1
Cat	1
Cat	1

Reducer 1

Key	Value
Dog	1
Pig	1

Reducer 2

No. of Reducers You Need

It is customisable according to your design.

By default, a single reducer is used in the program to gather all results together in the same place.

With a single reducer, it collects all key-value pairs in sorted order.

- Handy for composing the sorted result.
- Potentially resource heavy for the node running the reducer.
- Taking a long time to run.

Jobs which Require a Single Reducer

- If a job needs to:
  - Output a single file.
- Alternatively, the *TotalOrderPartitioner* can be used.
  - Uses an externally generated file which contains information about intermediate key distribution.
  - Partitions data such that all keys which go to the first Reducer are smaller than any which go to the second.
  - In this way, multiple Reducers can be used.
  - Concatenating the Reducers' output file results in a totally ordered list.

Jobs which Require a Fixed Number of Reducers

Some jobs will require a specific number of Reducers

Examples

- **Gathering data by Month, by Weekday, etc.**

Jobs with a Variable Number of Reducers

Many jobs can be run with a variable number of reducers.

Developer must decide how many to specify.

- Distributing the workload.

Typical way to determine how many reducers to specify:

- Test the job with a relatively small test dataset.
- Extrapolate to calculate the amount of intermediate data expected from the "real" input data.
- Use that to calculate the number of Reducers which should be specified.

Jobs with a Variable Number of Reducers

Drive Through

Drive Through 1

Drive Through 2

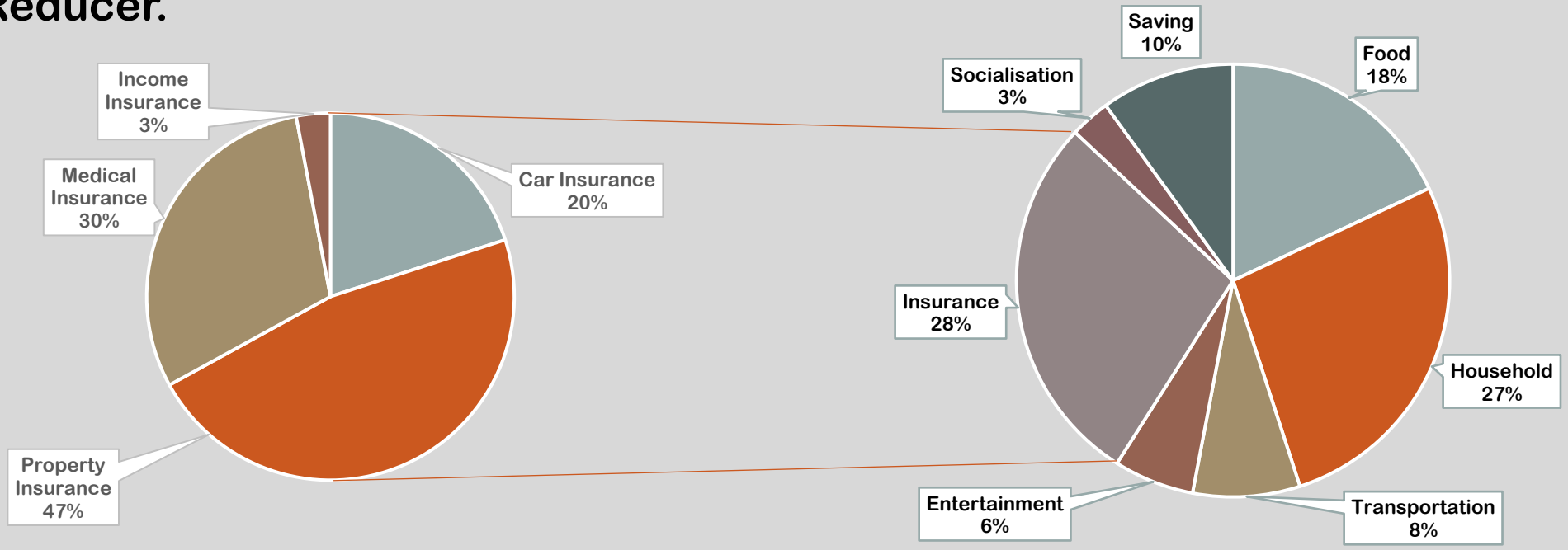
Estimate the number of Reduce slots likely to be available on the cluster.

- If more reducers than the capacity are specified, the total waiting time can be longer.

WRITE CUSTOMISED PARTITIONERS

Custom Partitioners

- In some cases, you'll prefer to write your own Partitioner
- For example, your key is a custom *WritableComparable* which contains a pair of values (*a*, *b*).
  - You may decide that all keys with the same value for *a* need to go to the same Reducer.



Custom Partitioners

Custom Partitioners are needed when performing a secondary sort.

Custom Partitioners are also useful to avoid potential performance issues.

- Avoiding one reducer having to deal with many very large lists of values.

```java
import org.apache.hadoop.mapreduce.Partitioner;

public class YearPartitioner<K2, V2> extends Partitioner<Text, Text> implements
    Configurable {

    private int reducer;

      public int getPartition(Text key, Text value, int numReduceTasks) {
            // determine reducer number between 0 and numReduceTasks-1
            //...
            return reducer;
          }
```

Creating a Custom Partitioner

- Create a class that extends Partitioner
- Override the *getPartition* method
  - Return an integer between 0 and one less than the number of Reducers.

Using a Custom Partitioner

- Specify the custom Partitioner in your driver code.

```
job.setPartitionerClass(MyPartitioner.class);
```

- If you need to set up variables in your partitioner, implement the *Configurable*.
  - If a Hadoop object implements *Configurable*, its *setConf()* method will be called once, when it is instantiated.
  - You can therefore set up variables in the *setConf()* method, which your *getPartition()* method will then be able to access.

```
// Defining the input/output paths.
FileInputFormat.setInputPaths(job, new Path(args[0]));
FileOutputFormat.setOutputPath(job, new Path(args[1]));

// Set the Mapper and the Reducer.
job.setMapperClass(LogFileMapper.class);
job.setReducerClass(LogFileReducer.class);
job.setPartitionerClass(YearPartitioner.class);
job.setNumReduceTasks(3);

// Intermediate output key/value produced by Mapper.
job.setMapOutputKeyClass(Text.class);
job.setMapOutputValueClass(IntWritable.class);

// Reducer output key/value class
job.setOutputKeyClass(Text.class);
job.setOutputValueClass(IntWritable.class);

boolean success = job.waitForCompletion(true);
System.exit(success ? 0 : 1);
```

SETTING UP VARIABLES FOR YOUR PARTITIONER

DRIVER CODE

LogFileMapper.java | ProcessLogs.java | YearPartitioner.java

```java
package stubs;

import java.util.HashMap;

import org.apache.hadoop.io.Text;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.conf.Configurable;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.mapreduce.Partitioner;

public class YearPartitioner<K2, V2> extends Partitioner<Text, IntWritable> implements
    Configurable {

  private Configuration configuration;
  HashMap<String, Integer> years = new HashMap<String, Integer>();
  private String[] YearList = {"2009","2010","2011"};
  private String tg;
  private boolean found;
  private int tgv;

  /**
   * Set up the months hash map in the setConf method.
   */
  @Override
  public void setConf(Configuration configuration) {
      for (int y = 0; y < YearList.length; y++) {
          years.put(YearList[y], y);
      }
  }
  @Override
  public Configuration getConf() {
    return configuration;
  }
  public int getPartition(Text key, IntWritable value, int numReduceTasks) {

      ///*
     tg = key.toString();
     found = false;
     for (int y = 0; y < YearList.length; y++) {
         if (tg.equals(YearList[y])) {
             found = true;
             tgv = y;
             break;
         }
     }

     if (found) {
         return tgv;
     }
     else {
         return 0;
     }
  }
}
```

SETTING UP VARIABLES FOR YOUR PARTITIONER

PARTITIONER

DATA INPUT AND OUTPUT

Data Types in Hadoop

Writable

Define a de/serialisation protocol.

Every data in Hadoop is *Writable*.

WritableComparable

Defines a sort order.

All keys must be *WritableComparable*.

IntWritable, LongWritable, Text, …

Concrete classes for different data types.

“Box” Classes in Hadoop

Hadoop’s built-in data type are “box” classes.

- They contain a single piece of data:
  - Text: string
  - IntWritable: int
  - LongWritable: long
  - FloatWritable: float
  - Etc.

Writable defines the wire transfer format.

- How the data is serialised and deserialised.

Creating a Complex *Writable*

- Assume we want a tuple (a, b) as the value emitted from a Mapper.
  - We could artificially construct it by:

```
Text t = new Text(a + “,” + b);
…
String[] arr = t.toString().split(“,”);
```

- Inelegant
- Problematic
  - Ex: *a* or *b* contained commas.
- Not always practical
  - Doesn’t easily work for binary objects.
- Solution: create your own *Writable* object.

The *Writable* Interface

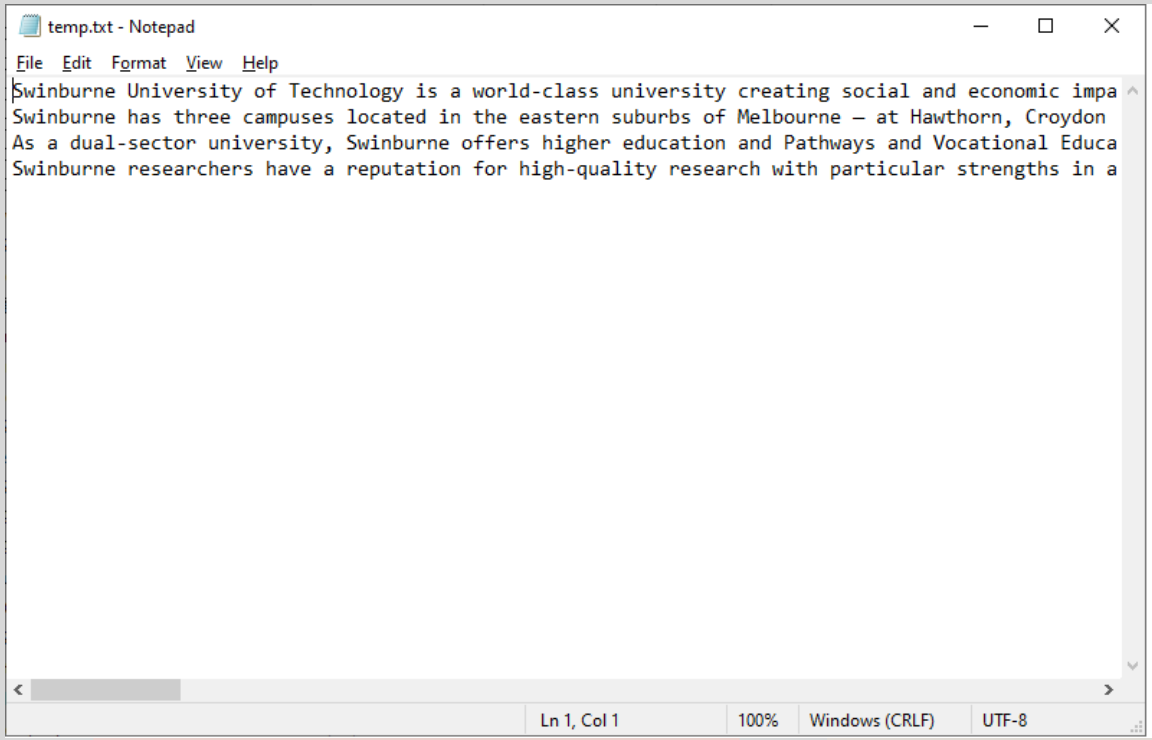
```
public interface Writable {
   void readFields(DataInput in);
   void write(DataOutput put);
}
```

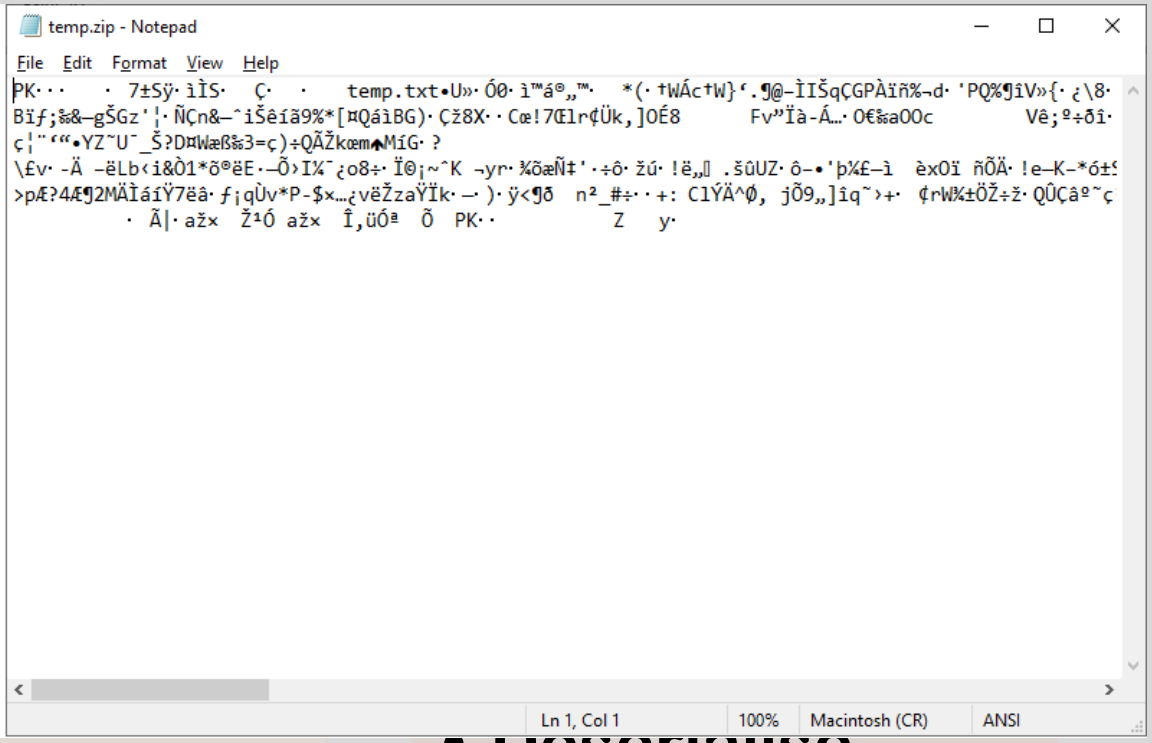
- The *readFields()* and *write()* methods will define how your custom object will be serialised and deserialised by Hadoop.
- The *DataInput* and *DataOutput* classes support
  - *boolean*
  - *byte, char* (Unicode: 2 bytes)
  - *double, float, int, long*
  - *String* (Unitcode or UTF-8)
  - Line until line terminator
  - unsigned *byte, short*
  - *byte* array

Example Custom *Writable*: DateWritable

```
class DateWritable implements Writable {
  int month, day, year;

  // Constructors omitted for brevity

  public void readFields(DataInput in) throws IOException {
    this.month = in.readInt();
    this.day = in.readInt();
    this.year = in.readInt();
  }

  public void write(DataOutput out) throws IOException {
    out.writeInt(this.month);
    out.writeInt(this.day);
    out.writeInt(this.year);
  }
}
```

Binary Objects





- Deserialise object

Binary Objects

Solution:

- Use byte arrays.

Write idiom:

- Serialise object to byte array
- Write byte count
- Write byte array

Read idiom:

- Read byte count
- Create byte array of proper size
- Read byte array
- Deserialise object

WritableComparable

WritableComparable is a sub-interface of Writable.

- **Must implement *compareTo*, *hashCode*, *equals* methods.**

All keys in MapReduce must be WritableComparable.

Making *DateWritable* a *WritableComparable*

```
class DateWritable implements WritableComparable<DateWritable> {
 int month, day, year;

 // Constructors omitted for brevity
 public void readFields (DataInput in) …
 public void write (DataOutput out) …

 public Boolean equals(Object o) {
  if (o instanceof DateWritable) {
   DateWritable other = (DateWritable) o;
   return this.year == other.year && this.month == other.month && this.day == other.day;
  }
  return false;
 }
```

Making *DateWritable* a *WritableComparable*

```
public int compareTo(DateWritable other) {
  /* return -1: if this data is earlier, 0: if dates are equal, 1: if this date is later */
  if (this.year != other.year) {
    return (this.year < other.year ? -1 : 1);
  } else if (this.month != other.month) {
    return (this.month < other.month ? -1 : 1);
  } else if (this.day != other.day) {
    return (this.day < other.day ? -1: 1);
  }
  return 0;
}

public int hashCode() {
  int seed = 123;
  return this.year * seed + this.month * seed + this.day * seed;
}
}
```

Using Custom Types in MapReduce Jobs

- Use methods in *Job* to specify your custom key/value types.
- For output of Mappers:

```
job.setMapOutputKeyClass(…);
job.setMapOutputValueClass(…);
```

- For output of Reducers:

```
job.setOutputKeyClass(…);
job.setOutputValueClass(…);
```

- Input types are defined by *InputFormat*

SAVING BINARY DATA USING SEQUENCEFILES AND AVRO DATA FILES

SequenceFiles

SequenceFiles are files containing binary-encoded key-value pairs

- Work naturally with Hadoop data types.
- SequenceFiles include metadata which identifies the data types of the key and value.

Three file types in one.

- Uncompressed
- Record-compressed
- Block-compressed

Often used in MapReduce

- Especially when the output of one job will be used as the input for another
  - SequenceFileInputFormat
  - SequenceFileOutputFormat

Directly Accessing SequenceFiles

- It is possible to directly access *SequenceFiles* from your code:

```
Configuration config = new Configuration();
SequenceFile.Reader reader
  = new SequenceFile.Reader(FileSystem.get(config), path, config);

Text key = (Text) reader.getKeyClass().newInstance();
IntWritable value = (IntWritable) reader.getValueClass().newInstance();

While (reader.next(key,value)) {
  // do something here
}

Reader.close();
```

Strong Points and Problems with SequenceFiles

- Because they are binary, they have faster read/write than text formatted files.
- They are most typically accessible via Java API only.
- If the definition of the key or value object changes, the file becomes unreadable.
- Small file may cause memory overhead, but can be avoided with proper arrangements.
- Record-based file is much less than Block-based file, this causes the overhead cost and spins up more mappers

An Alternative to SequenceFiles: Avro

Apache Avro is a serialisation format which is becoming a popular alternative to SequenceFiles.

Self-describing file format.

Compact file format.

Portable across multiple languages

- Support for C, C++, Java, Python, Ruby and others.

Compatible with Hadoop

- Via the *AvroMapper* and *AvroReducer* classes.

ISSUES TO CONSIDER WHEN USING FILE COMPRESSION

Hadoop and Compressed Files

Hadoop understands a variety of file compression formats.

If a compressed file is included as one of the files to be processed, Hadoop will automatically decompress it and pass the decompressed contents to the Mapper.

If the file is not in a “splittable file format,” it can only be decompressing by starting at the beginning of the file and continuing on to the end.

Non-Splittable File Formats in Hadoop

If the MapReduce framework receives a non-splittable file, it passes the *entire file* to a single Mapper.

This can result in one Mapper running for far longer than the others.

Typically it is not a good idea to use GZip to compress MapReduce input files.

Splittable Compression Formats: LZ4

LZ4 is a splittable compression format under conditions.

To make an LZ4 file splittable, you must first index the file.

The index file contains information about how to break the LZ4 file into splits that can be decompressed.

Common Compression Codecs Supported in Hadoop

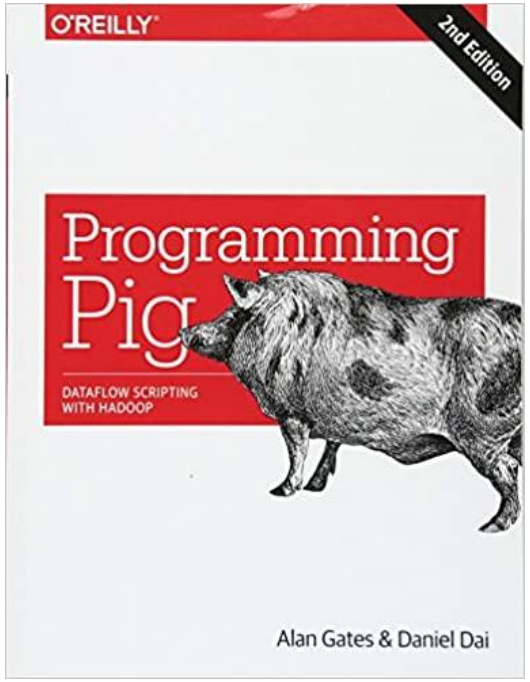
Codec	File Extension	Splitable?	Degree of Compression	Compression Speed
Gzip	.gz	No	Medium	Medium
Bzip2	.bz2	Yes	High	Slow
Snappy	.snappy	No	Medium	Fast
LZ4	.lz4	No, unless indexed	Medium	Fast

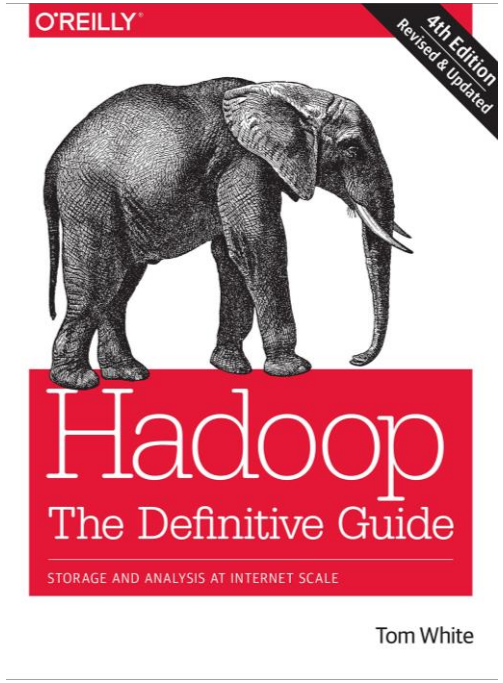
https://www.dummies.com/programming/big-data/hadoop/compressing-data-in-hadoop/

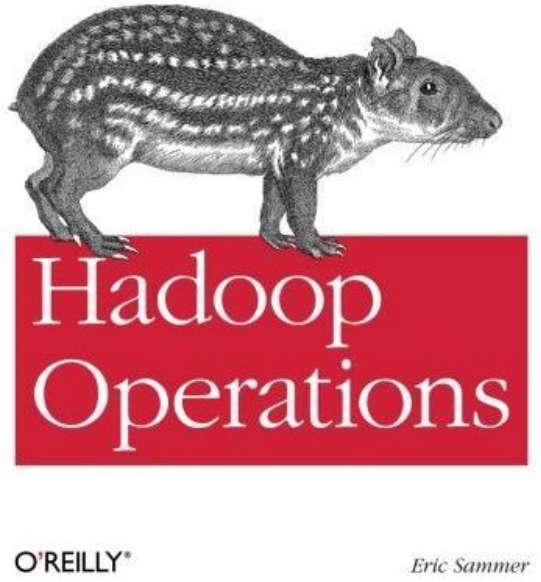
Compressing Output SequenceFiles with Snappy

- Specify output compression in the *Job* object.
- Specify block or record compression.
  - Block compression is recommended for the Snappy codec.
- Set the compression codec to the Snappy codec in the *Job* object.

```
import org.apache.hadoop.mapreduce.lib.output.SequenceFileOutputFormat;
import org.apache.hadoop.io.SequenceFile.CompressionType;
import org.apache.hadoop.io.compress.SnappyCodec;
. . .
job.setOutputFormatClass(SequenceFileOutputFormat.class);
FileOutputFormat.setCompressOutput(job,true);
FileOutputFormat.setOutputCompressorClass(job,SnappyCodec.class);
SequenceFileOutputFormat.setOutputCompressionType(job,
  CompressionType.BLOCK);
```







Texts and Resources

- Unless stated otherwise, the materials presented in this lecture are taken from:
  - Hadoop: the definitive guide, White, Tom (Tom E.) author., 4th ed.., Sebastopol, California: O'Reilly, 2015.
  - Hadoop operations, Sammer, Eric., Loukides, Michael Kosta.; Nash, Courtney.; Romano, Robert (Illustrator), illustrator., First edition., Sebastopol, CA: O'Reilly, 2012.
  - Programming pig: dataflow scripting with hadoop, Gates, Alan, author.; Dai, Daniel, 2nd edition., Beijing, China: O'Reilly, 2017

Unit Summary

- MapReduce Code Explanation.